

1. Resolución de Ejercicios Propuestos

Desarrollo de las soluciones para los ejercicios propuestos de la Actividad 6. Esta sección aborda los principios fundamentales de la Programación Orientada a Objetos: sobrecarga (de métodos y constructores), polimorfismo dinámico y la implementación de clases abstractas.

1.1. Descripción de los Ejercicios

A continuación se detallan las lógicas implementadas para los diferentes ejercicios:

- **Ejercicio 2.10 (Sobrecarga de Métodos):** Se implementó la clase Suma demostrando cómo una clase puede tener múltiples versiones de un mismo método cambiando la signatura. Se definieron cuatro operaciones de suma que aceptan y devuelven tanto enteros como decimales.
- **Ejercicio 2.11 (Sobrecarga de Constructores):** Se modelaron las clases Empleado y Caja. Para Empleado se proveen constructores vacíos (que asignan valores por defecto) y con parámetros. Para la clase Caja, se ilustró la optimización del código utilizando la palabra reservada `this()` para llamar a un constructor general desde constructores más específicos.
- **Ejercicio 4.4 y 4.6 (Polimorfismo):** Utilizando las clases Profesor y ProfesorTitular, se ejecutaron pruebas de escritorio. En el código se generó un Vector de profesores donde la máquina virtual invocó las funciones dinámicamente según la referencia instanciada en memoria, demostrando el comportamiento del enlazado tardío.
- **Ejercicio 4.7 (Clases Abstractas):** Se generó el contrato arquitectónico a través de la clase abstracta Numérica. Posteriormente, la clase concreta Fracción implementó toda la lógica aritmética heredada, asegurando la simplificación por MCD tras cada operación.

1.2. Diagramas de clases

Se presentan las relaciones, métodos y atributos modelados para los distintos ejercicios.

Ejercicio 2.10

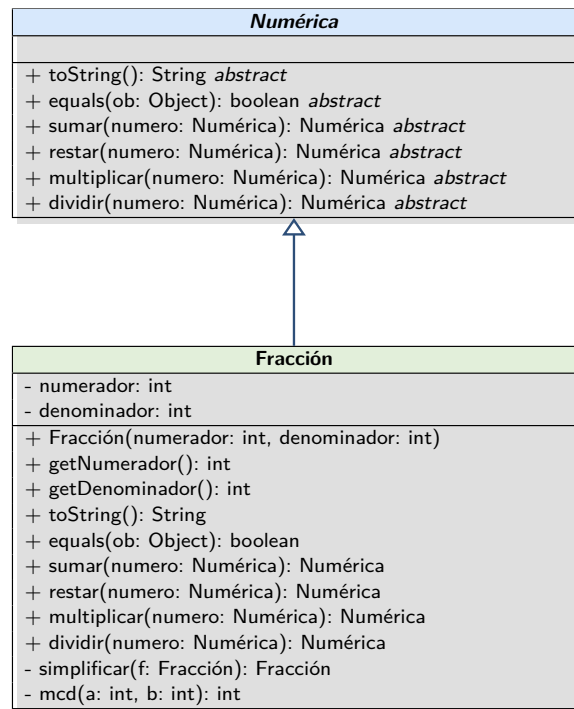
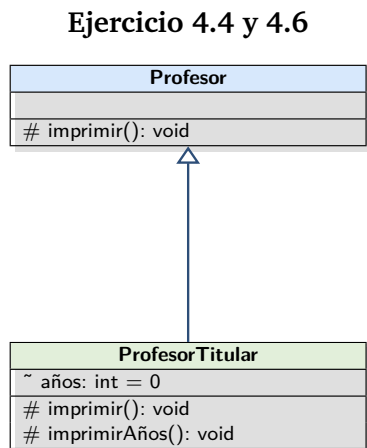
Suma
<pre>+ sumar(a: int, b: int): int + sumar(a: int, b: int, c: int): int + sumar(a: double, b: double): double + sumar(a: double, b: double, c: double): double + main(args: String[])\$</pre>

Ejercicio 2.11

Empleado
<pre>- identificador: int - nombre: String - apellidos: String - edad: int + Empleado() + Empleado(identificador: int, nombre: String, apellidos: String, edad: int) + imprimir(): void + main(args: String[])\$</pre>

Caja
<pre>- base: double - anchura: double - altura: double - tipo: String + Caja(base: double, anchura: double, altura: double) + Caja() + Caja(longitud: double) + Caja(base: double, anchura: double, altura: double, tipo: String) + imprimir(): void + main(args: String[])\$</pre>

Ejercicio 4.7



1.3. Casos de uso y Resultados (Pruebas)

- **Ejercicio 2.10 y 2.11:** La ejecución retorna los valores instanciados y sumados correspondientes a las cajas, empleados y operaciones, asegurando la sobrecarga exitosa sin errores de compilación.
- **Ejercicio 4.4 y 4.6:** El casting se ejecutó con éxito. La salida del sistema, demostrando polimorfismo dinámico, fue “*Es un profesor.*” y seguido “*Es un profesor titular.*”
- **Ejercicio 4.7:** Los resultados probaron igualdad verdadera cruzando las fracciones ‘1/2’ y ‘2/4’. Además las sumas, multiplicaciones y divisiones generaron nuevas fracciones con el numerador y denominador correctos y simplificados.

1.4. Links a los Repositorios

Para verificar el funcionamiento y el código fuente respectivo, se debe ejecutar cada archivo Java alojado en el repositorio de GitHub:

- **Ejercicio 2.10:** https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_10
- **Ejercicio 2.11:** https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_11
- **Ejercicio 4.4:** https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_4
- **Ejercicio 4.6:** https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_6
- **Ejercicio 4.7:** https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_7